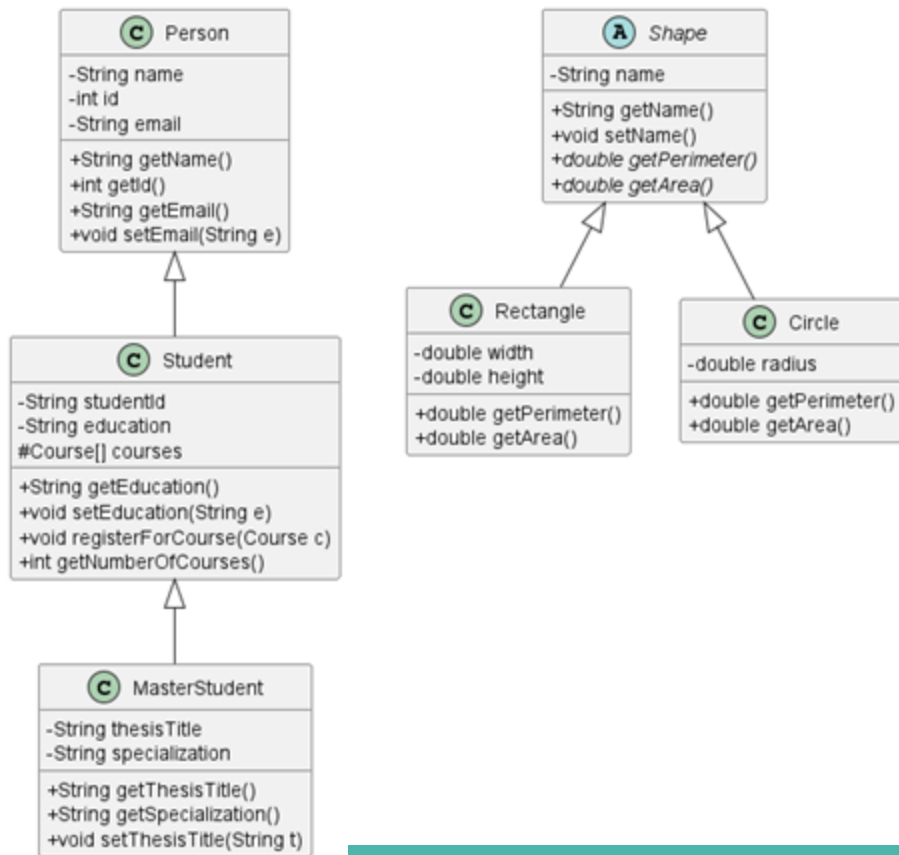

Interfaces

Lukas Holik

lukasholik@cs.aau.dk

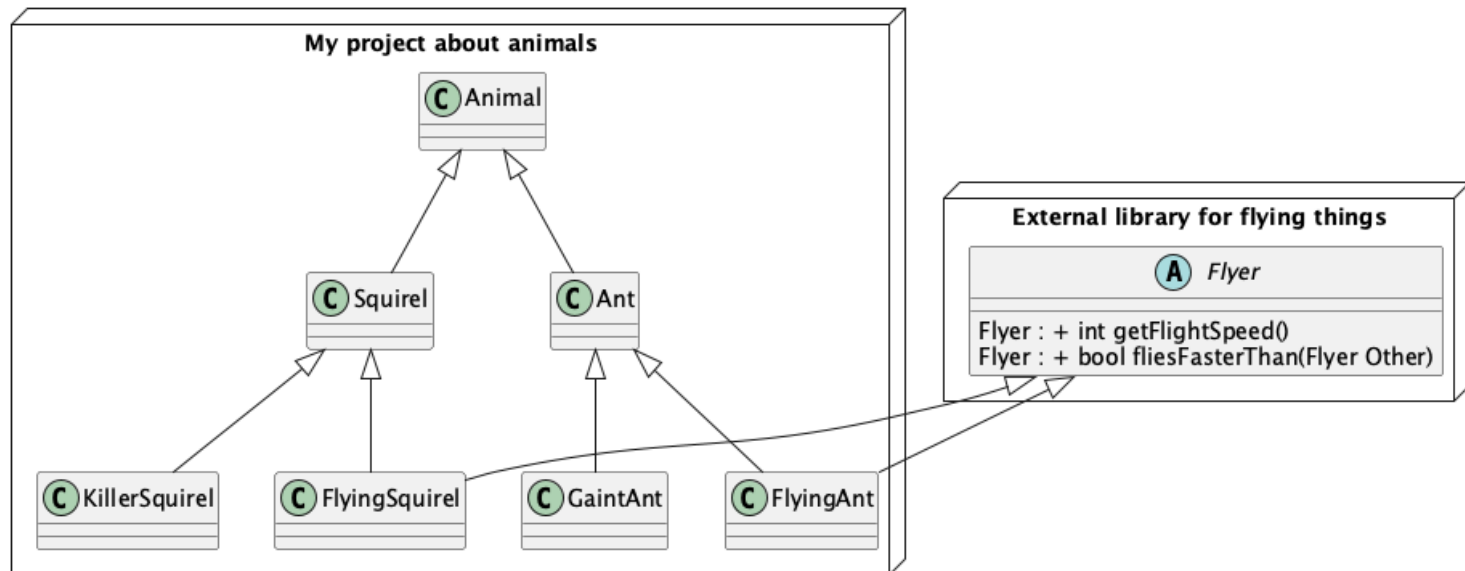
(based on slides of Gabriela Montoya, thanks)

Classes and Inheritance

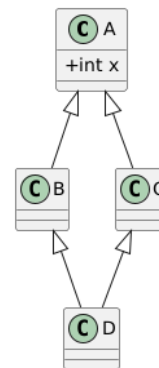


```
public class Rectangle extends Shape {
    private double width;
    private double height;
    public Rectangle(double w, double h) {
        super("Rectangle");
        this.width = w;
        this.height = h;
    }
    @Override
    public double getPerimeter() {
        return 2.0 * (width + height);
    }
    @Override
    public double getArea() {
        return width * height;
    }
}
```

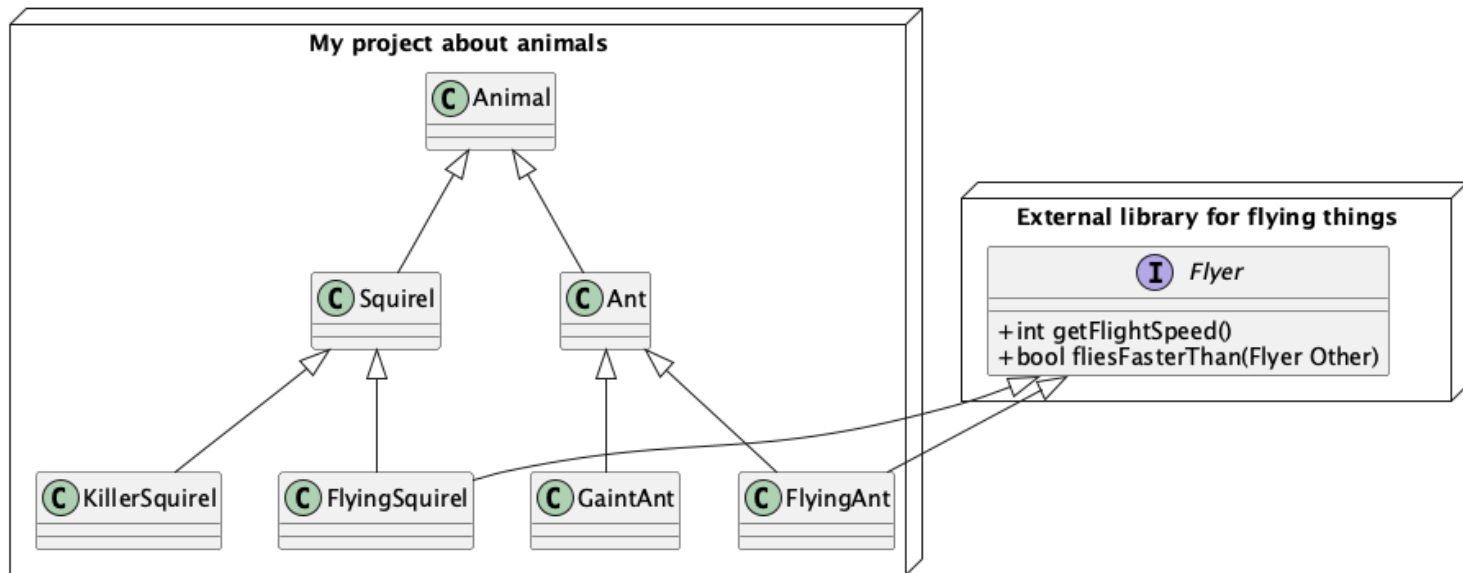
Motivation



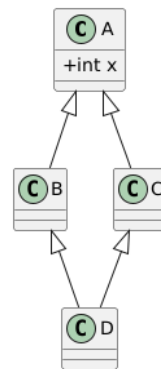
- Can't be done! A class has **at most one superclass**.
- Java prohibits multiple class inheritance, because it is messy: **diamonds**



Motivation



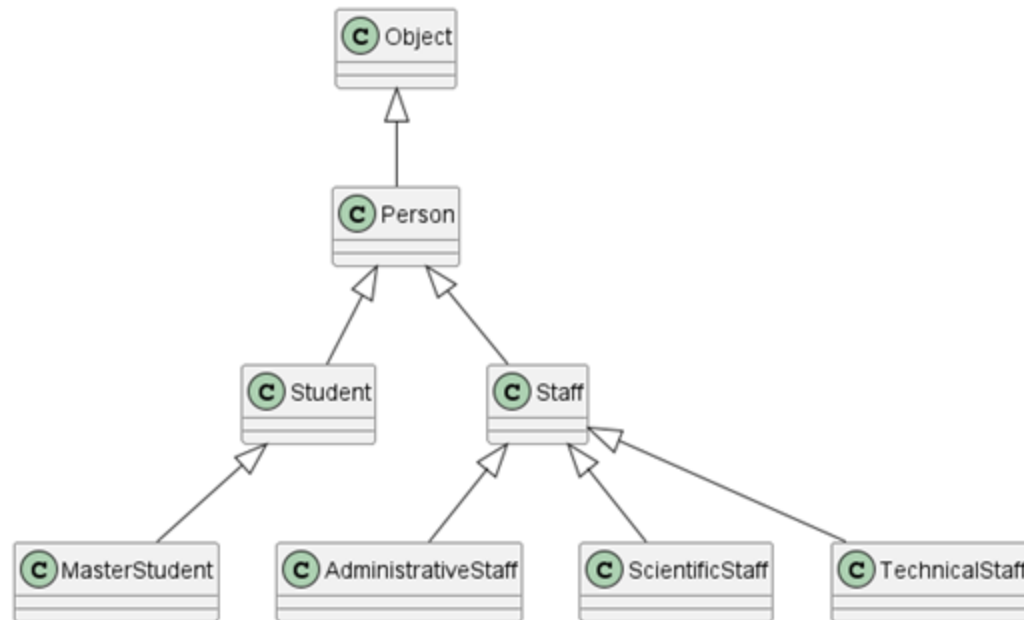
- Java solution is **interfaces**.
- Something as purely abstract class **without fields** (prevents diamond), and **allowed multiple inheritance** from it.



Persons and courses ...

Interfaces to combine new types with the classes in the hierarchy?

Identifiable, Ageable, Nameable,...



Identifiable

An **interface** defines a **specification**:
an abstract type and its operations.

```
interface Identifiable {  
    int getIdentifier();  
}
```

```
static void showIds(Identifiable[] is) {  
    for (Identifiable e : is) {  
        int id = e.getIdentifier();  
        System.out.println(id);  
    }  
}
```

```
class Person implements Identifiable {  
    private String name;  
    private int id;  
    ...  
    @Override  
    public int getIdentifier() {  
        return id;  
    }  
}
```

```
Person p1 = new Person("Jens", 123);  
Person p2 = new Person("Mark", 124);  
  
Identifiable[] elems = { p1, p2 };  
  
showIds(elems);
```

Members of an interface

- Members are by default **public**.
 - But private methods are allowed.
- Fields are **static** and **final**.
 - And must be initialized at declaration time.
- Methods are by default **abstract**.

```
interface Identifiable {  
    int DEFAULT = 0;  
    int getIdentifier();  
}
```



```
interface Identifiable {  
    public static final int DEFAULT = 0;  
    public abstract int getIdentifier();  
}
```

Can Course implement Identifiable?

```
interface Identifiable {  
    int getIdentifier();  
}
```

```
class Course {  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    ...  
}
```


Person and Course implement Identifiable<T>

```
interface Identifiable<T> {  
  
    T getIdentifier();  
  
}
```

```
class Person implements Identifiable<Integer>  
{  
    private String name;  
    private int id;  
    ...  
    @Override  
    public Integer getIdentifier() {  
        return id;  
    }  
}
```

```
class Course implements Identifiable<String>  
{  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    @Override  
    public String getIdentifier() {  
        return code;  
    }  
}
```

Can we add a new method to Identifiable?

```
interface Identifiable<T> {  
    boolean checkIdentifier(T id);  
    T getIdentifier();  
}
```

```
class Course implements Identifiable<String>  
{  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    @Override  
    public String getIdentifier() {  
        return code;  
    }  
}
```

Course does not compile!

Default methods allow for interfaces that evolve

```
interface Identifiable<T> {  
  
    default boolean checkIdentifier(T id) {  
        return getIdentifier().equals(id);  
    }  
  
    T getIdentifier();  
}
```

default methods

- **Instance** method providing a default implementation
- Can be overridden

```
class Course implements Identifiable<String>  
{  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    @Override  
    public String getIdentifier() {  
        return code;  
    }  
}
```

Concrete classes **must override all** the **abstract** methods inherited from the interface they implement.¹¹

Static methods in Interfaces

- Can be only called using the interface as context.
- Are **not inherited**.

```
public interface Identifiable<T> {  
    T getIdentifier();  
    public static<T> void showIds(Identifiable<T>[] is) {  
        for (Identifiable<T> e : is) {  
            T id = e.getIdentifier();  
            System.out.println(id);  
        }  
    }  
}
```

```
Course c = new Course("OOP",  
    "Object-oriented programming");  
Person p = new Person("Jens", 123);  
Identifiable[] elems = { c, p };  
Identifiable.showIds(elems);
```

Nested types

- Nested interfaces are **by default public and static**.
- Are accessed through their enclosing interfaces.
- Their private members are accessible by the enclosing type.

```
public interface Collection {  
    boolean add(Elem e);  
    boolean find(Elem e);  
    boolean remove(Elem a);  
  
    public static interface Elem {  
        String getKey();  
        String getName();  
    }  
}
```

```
Collection.Elem e = new Course("OOP", "bla... ");
```

```
class Course implements Collection.Elem {  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    @Override  
    public String getKey() {  
        return code;  
    }  
    @Override  
    public String getName() {  
        return title;  
    }  
}
```

What is the type of the following expression?

- a) Object a;
- b) Identifiable b;
- c) Course c;
- d) MasterCourse d;

Which of the variables declared above can be used instead of **?1** in the following statement?

?1 = new Course("OOP", "Object-oriented prog.");

```
class Course implements Identifiable<String> {
    private String code;
    private String title;
    private ArrayList<String> goals;

    public Course (String c, String t) {
        this.code = c;
        this.title = t;
        this.goals = new ArrayList<String>();
    }
    public void addGoal(String g) {
        goals.add(g);
    }
    public void removeLastGoal() {
        goals.remove(goals.size()-1);
    }
    @Override
    public String getIdentifier() {
        return code;
    }
    @Override
    public int hashCode() {
        return code.hashCode();
    }
}
```

```
interface Identifiable<T> {  
    T getIdentifier();  
}
```

```
Identifiable b = new Course("OOP",  
    "Object-oriented programming");
```

Can we invoke the method **addGoal** on the object referenced by **b**?

- a) No
- b) Yes

```
class Course implements Identifiable<String> {  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    public void addGoal(String g) {  
        goals.add(g);  
    }  
    public void removeLastGoal() {  
        goals.remove(goals.size()-1);  
    }  
    @Override  
    public String getIdentifier() {  
        return code;  
    }  
    @Override  
    public int hashCode() {  
        return code.hashCode();  
    }  
}
```

```
((Course) b).addGoal("Use correct terminology");
```

Please provide your answers using [this form](#)

```
interface Identifiable<T> {  
    T getIdentifier();  
}
```

Object **a** = new Course("OOP",
"Object-oriented programming");

How many methods can be called on **a** (with **a** as context)?

- a) At most two
- b) Between three and five
- c) **Between six and nine**
- d) More than nine

```
class Course implements Identifiable<String> {  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    public void addGoal(String g) {  
        goals.add(g);  
    }  
    public void removeLastGoal() {  
        goals.remove(goals.size()-1);  
    }  
    @Override  
    public String getIdentifier() {  
        return code;  
    }  
    @Override  
    public int hashCode() {  
        return code.hashCode();  
    }  
}
```



```
interface Identifiable<T> {  
    T getIdentifier();  
}
```

```
Identifiable b = new Course("OOP",  
    "Object-oriented programming");
```

How many methods can be called on **b** (with **b** as context)?

- a) At most two
- b) Between three and five
- c) Between six and nine
- d) More than nine

```
class Course implements Identifiable<String> {  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    public void addGoal(String g) {  
        goals.add(g);  
    }  
    public void removeLastGoal() {  
        goals.remove(goals.size()-1);  
    }  
    @Override  
    public String getIdentifier() {  
        return code;  
    }  
    @Override  
    public int hashCode() {  
        return code.hashCode();  
    }  
}
```

```
interface Identifiable<T> {  
    T getIdentifier();  
}
```

```
Course c = new Course("OOP",  
    "Object-oriented programming");
```

How many methods can be called on **c** (with **c** as context)?

- a) At most two
- b) Between three and five
- c) Between six and nine
- d) More than nine

```
class Course implements Identifiable<String> {  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    public void addGoal(String g) {  
        goals.add(g);  
    }  
    public void removeLastGoal() {  
        goals.remove(goals.size()-1);  
    }  
    @Override  
    public String getIdentifier() {  
        return code;  
    }  
    @Override  
    public int hashCode() {  
        return code.hashCode();  
    }  
}
```

Does the following class compile?

```
interface Identifiable<T> {  
    T getIdentifier();  
}
```

```
class Course implements Identifiable<String> {  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    @Override  
    String getIdentifier() {  
        return code;  
    }  
}
```

```
public interface Identifiable<T> {  
  
    T getIdentifier();  
  
}
```

```
public interface Cloneable {  
  
}
```

```
public interface Comparable<T> {  
  
    int compareTo(T o);  
  
}
```

Course is a subtype of:

- Identifiable
- Cloneable
- Comparable
- Object

```
public class Course implements Identifiable<String>,  
                                Cloneable, Comparable<Course> {  
  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    public void addGoal(String g) {  
        goals.add(g);  
    }  
    @Override  
    public String getIdentifier() {  
        return code;  
    }  
    @Override  
    public Course clone() {  
        Course c = new Course(code, title);  
        for (String g : goals) {  
            c.addGoal(g);  
        }  
        return c;  
    }  
    @Override  
    public int compareTo(Course other) {  
        return code.compareTo(other.code);  
    }  
}
```

Members inherited from another interface

- An interface inherits abstract and default methods from its superinterface.
- Static and private methods are not inherited.

```
public interface Identifiable<T> {  
    T getIdentifier();  
  
    default boolean checkIdentifier(T id) {  
        return getIdentifier().equals(id);  
    }  
  
    public static<T> void showIds(Identifiable<T>[] is) {  
  
        for (Identifiable<T> e : is) {  
            T id = e.getIdentifier();  
            System.out.println(id);  
        }  
    }  
}
```

```
public interface Vehicle extends Identifiable<String> {  
    String getBrand();  
    String getModel();  
    int getYear();  
}
```

Interfaces and multiple inheritance

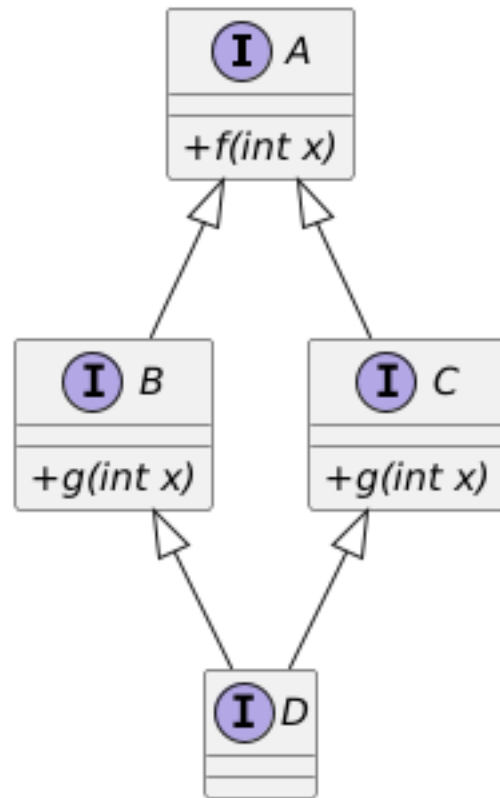
- One method can be inherited from several interfaces.
- The diamond is possible with abstract methods, and it is ok.

```
interface A {  
    int f(int x);  
}
```

```
interface B extends A {  
    int g(int x);  
}
```

```
interface C extends A {  
    int g(int x);  
}
```

```
interface D extends B,C {  
    int g(int x);  
}
```



Interfaces and multiple inheritance

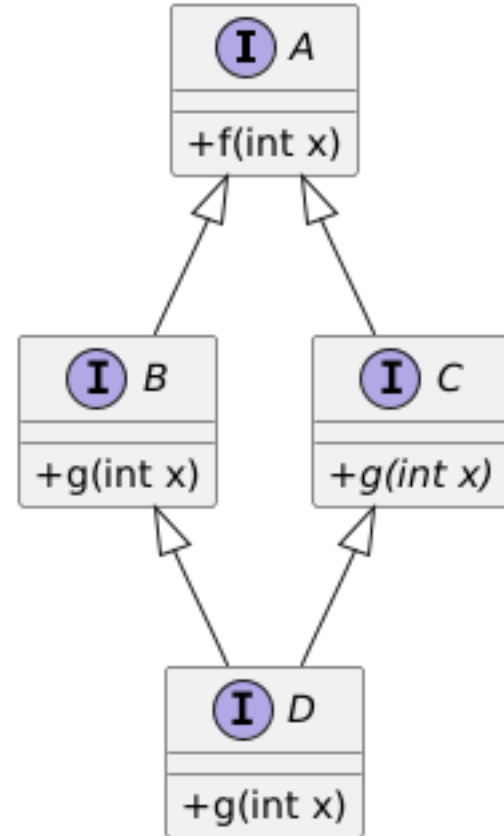
- Non-abstract methods inherited from multiple interfaces must be overridden.

```
interface A {  
    default int f(int x) {  
        return 1;  
    }  
}
```

```
interface B extends A {  
    default int g(int x) {  
        return 2;  
    }  
}
```

```
interface C extends A {  
    int g(int x);  
}
```

```
interface D extends B,C {  
    @Override  
    default int g(int x) {  
        return 3;  
    }  
}
```



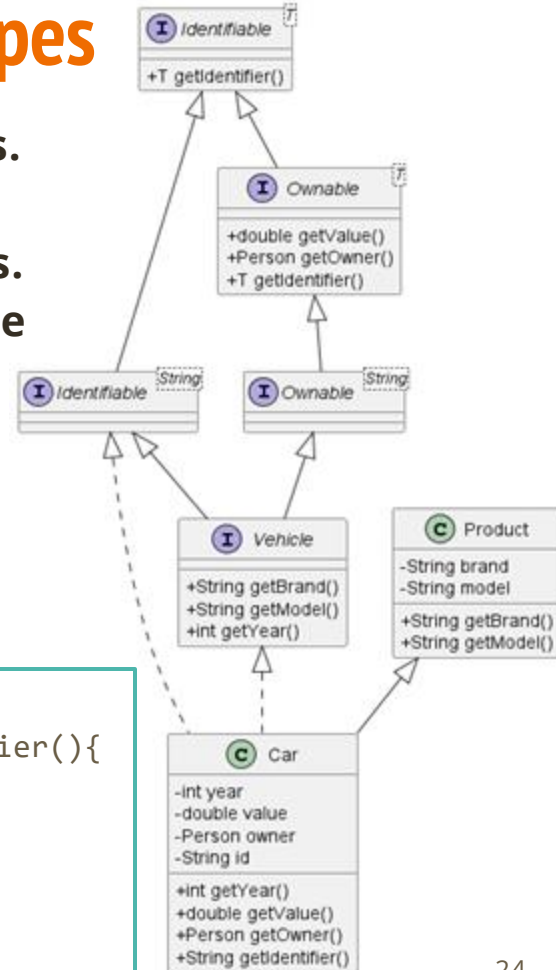
Inheriting a method from multiple supertypes

```
interface Identifiable<T> {  
    T getIdentifier();  
    default boolean checkIdentifier(T id) {  
        return getIdentifier().equals(id);  
    }  
}
```

```
interface Vehicle extends  
Identifiable<String>, Ownable<String> {  
    String getBrand();  
    String getModel();  
    int getYear();  
    default String getIdentifier() {  
        return Ownable.super.getIdentifier();  
    }  
}
```

1. The superclass wins.
2. The most specific superinterface wins.
3. The method must be overridden.

```
interface Ownable<T> {  
    default T getIdentifier(){  
        return null;  
    }  
    double getValue();  
    Person getOwner();  
}
```



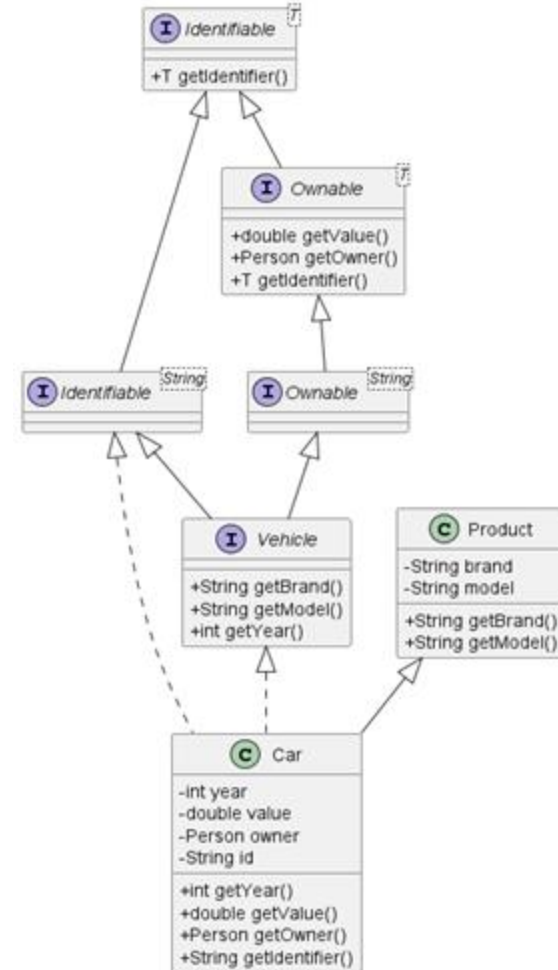
Possible compile-time type of c?

```
Person jens = ...
```

```
?1 c = new Car(2000, 100000, jens, "FF33333");
```

- a) Car
- b) Product
- c) Ownable<String>
- d) Ownable<Integer>
- e) Object

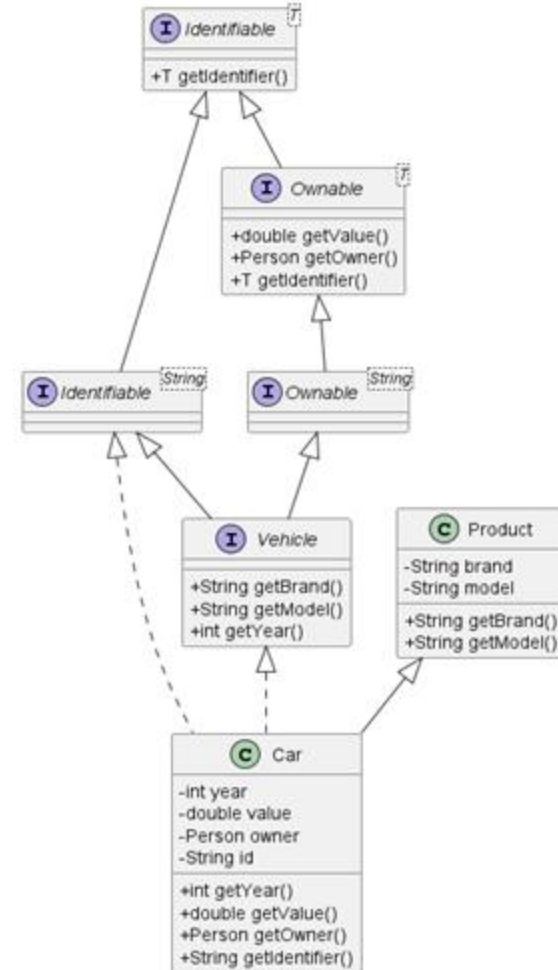
Please provide your answers using [this form](#)



Possible runtime type of v?

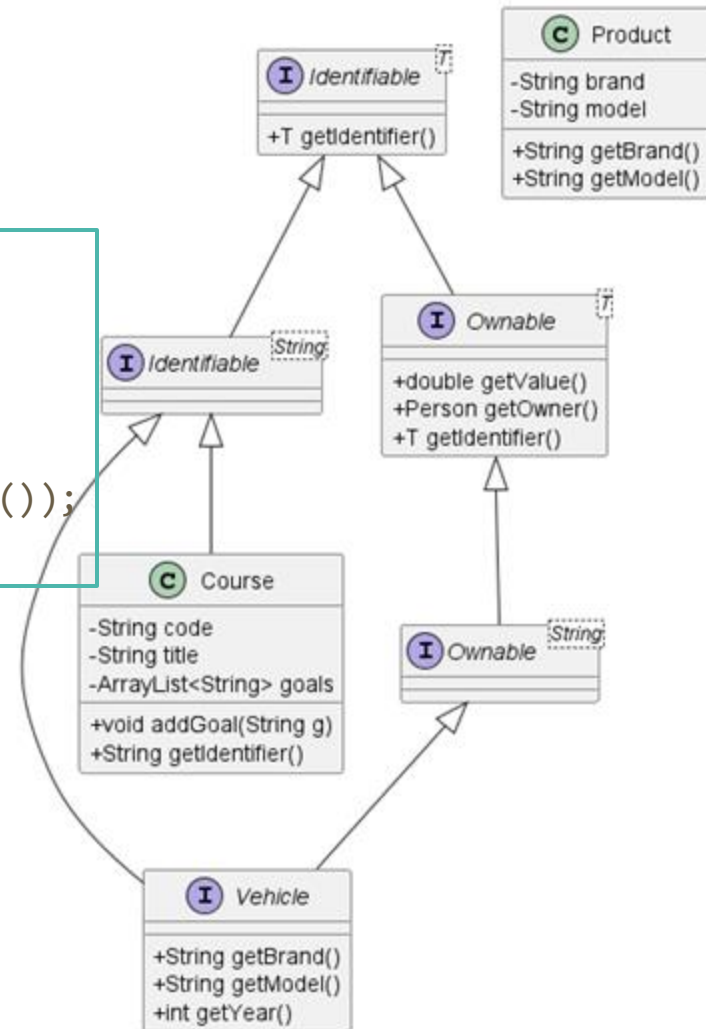
```
Vehicle v = new ?2(...);
```

- a) Car
- b) Product
- c) Ownable<String>
- d) Ownable<Integer>
- e) Object



Does the compile-time check pass?

```
Course c = new Course("OOP",  
    "Object-oriented programming");  
...  
if (c instanceof Identifiable) {  
    Identifiable i = (Identifiable) c;  
    System.out.println("identifier:" + i.getIdentifier());  
}
```



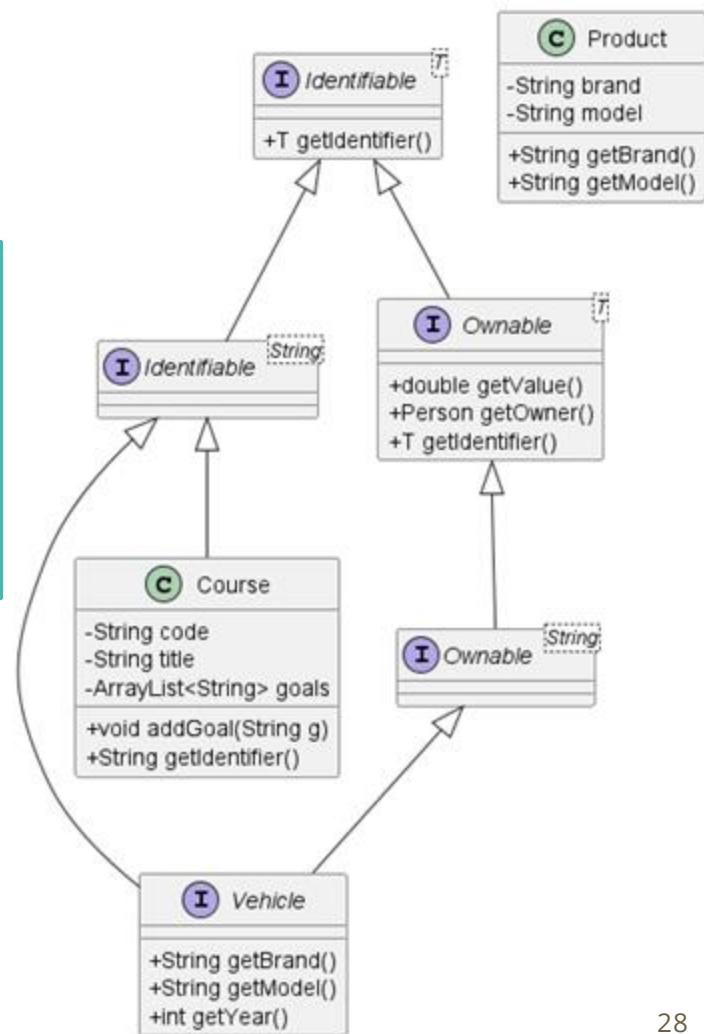
Does the compile-time check pass?

```
Course c = new Course("OOP",  
    "Object-oriented programming");  
...  
if (c instanceof Ownable) {  
    Ownable o = (Ownable) c;  
    System.out.println("value: "+o.getValue());  
}
```

Compiler does not look at the context of the entire code.

So it must allow for possibilities. Could the rest of the code contain something that would make the run-time check pass?

Yes! A class X which extends Course and implements Ownable.



Does the compile-time check pass?

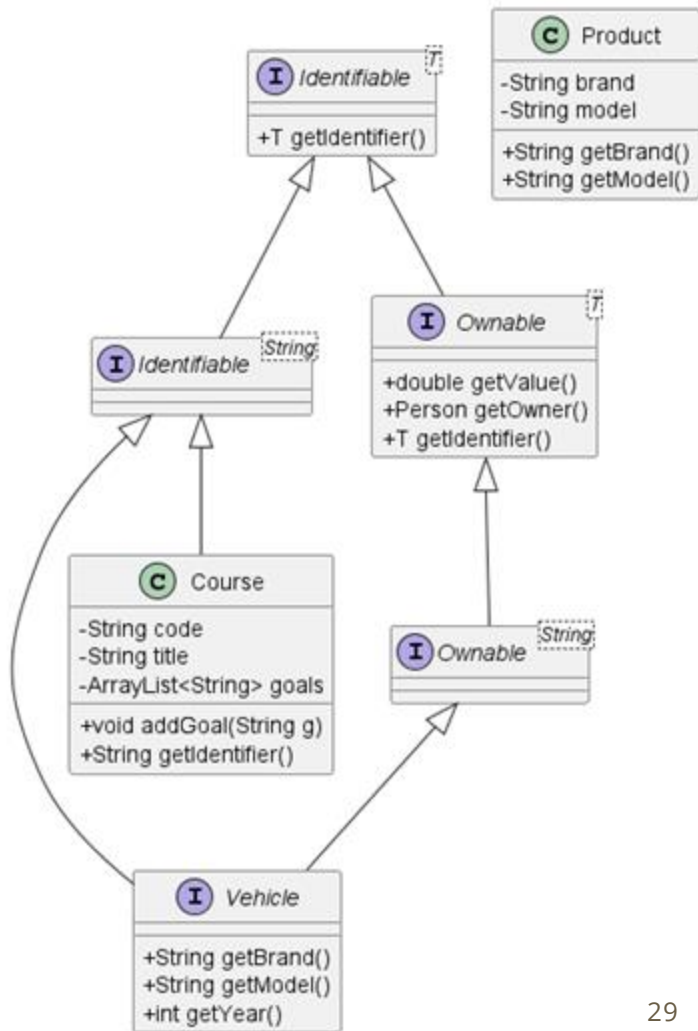
X

```
Course c = new Course("OOP",  
    "Object-oriented programming");  
...  
if (c instanceof Product) {  
    Product p = (Product) c;  
    System.out.println("model: "+p.getModel());  
}
```

Could the rest of the code contain something that would make the run-time check pass?

No! For the check to pass, Product would have to be a descendant of Course (direct or indirect). But the compiler can directly see that Product does not extend anything (besides Object).

Please provide your answers using [this form](#)



Marker Interface

- A marker interface has no members.
- Its meaning depends on the context.
- Cloneable indicates that the class is meant to provide an implementation of clone() that returns a field-for-field copy.

```
public interface Cloneable {}
```

```
public class Course implements Cloneable {
    private String code;
    private String title;
    private ArrayList<String> goals;

    public Course (String c, String t) {
        this.code = c;
        this.title = t;
        this.goals = new ArrayList<String>();
    }
    public void addGoal(String g) {
        goals.add(g);
    }
    @Override
    public Course clone() {
        Course c = new Course(code, title);
        for (String g : goals) {
            c.addGoal(g);
        }
        return c;
    }
    @Override
    public int compareTo(Course other) {
        return code.compareTo(other.code);
    }
}
```

Functional Interface

- A functional interface has exactly one abstract method.
- The number of default or static methods is irrelevant.

```
@FunctionalInterface
public interface Identifiable<T> {

    T getIdentifier();

}
```

```
@FunctionalInterface
public interface Ownable<T> {

    Person getOwner();

}
```

Is Vehicle a functional interface?

```
@FunctionalInterface
public interface Identifiable<T> {

    T getIdentifier();

}
```

```
public interface Ownable<T> extends Identifiable<T> {

    Person getOwner();

}
```

```
public interface Vehicle
    extends Identifiable<String>, Ownable<String> {

    String getBrand();
    String getModel();
    int getYear();

}
```

X

Is Ownable a functional interface?

```
@FunctionalInterface
public interface Identifiable<T> {

    T getIdentifier();

}
```

```
public interface Ownable<T> extends Identifiable<T> {

    Person getOwner();

}
```

X

Please provide your answer using [this form](#)

java.lang.Comparable

- `compareTo(Object)` is
 - **A negative integer** if this comes before the Object passed as parameter
 - **0** if this and the Object passed as parameter represent the same instance
 - **A positive integer** if this comes after the Object passed as parameter
- `Comparable<Course>` indicates that Course supports comparison with instances of Course

```
public interface Comparable<T> {  
  
    int compareTo(T o);  
  
}
```

```
public class Course implements Comparable<Course> {  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
    }  
    public void addGoal(String g) {  
        goals.add(g);  
    }  
    @Override  
    public int compareTo(Course other) {  
        return code.compareTo(other.code);  
    }  
}
```

java.util.Comparator

This interface has

- 2 abstract methods
- 9 static methods
- 7 default methods

```
interface Comparator<T> {  
    boolean equals(T obj);  
    boolean compare(T o1, T o2);  
    static <T> Comparator<T> nullsFirst(Comparator<? super T> comparator) { ... }  
    static <T> Comparator<T> nullsLast(Comparator<? super T> comparator) { ... }  
    static <T extends Comparable<? super T>> Comparator<T> naturalOrder() { ... }  
    default Comparator<T> reversed() { ... }  
    default Comparator<T> thenComparing(Comparator<? super T> other) { ... }  
    ...  
}
```

TitleComparator

```
public class TitleComparator implements Comparator<Course> {  
    @Override  
    public int compare(Course c1, Course c2) {  
        String title1 = c1.getTitle();  
        String title2 = c2.getTitle();  
        return title1.compareTo(title2);  
    }  
}
```

```
Course[] courses = { c1, c2, c3 };  
Comparator tComparator = new TitleComparator();  
Arrays.sort(courses, tComparator);  
System.out.println(Arrays.toString(courses));
```

codeTitleComparator

```
public class TitleComparator implements Comparator<Course> {  
    @Override  
    public int compare(Course c1, Course c2) {  
        String title1 = c1.getTitle();  
        String title2 = c2.getTitle();  
        return title1.compareTo(title2);  
    }  
}
```

```
Comparator tComparator = new TitleComparator();  
Comparator<Course> codeComparator = Comparator.naturalOrder();  
Comparator<Course> codeTitleComparator = codeComparator.thenComparing(tComparator);  
Arrays.sort(courses, codeTitleComparator);  
System.out.println(Arrays.toString(courses));
```

Enum types

- Allow for creating **ordered** lists of constants as types.

```
public enum Severity {  
    LOW, MEDIUM, HIGH, URGENT;  
}
```

ordinals:

0

1

2

3

names:

"LOW"

"MEDIUM"

"HIGH"

"URGENT"

```
Severity s1 = Severity.valueOf("HIGH");
```

```
int sOrd = s1.ordinal();  
String sName = s1.name();
```

```
Severity s2 = Severity.values()[0];  
boolean b = s2.equals(Severity.LOW);
```

```
int diff = s2.compareTo(Severity.HIGH);
```

Severity.values() returns an array with elements: Severity.LOW, Severity.MEDIUM, Severity.HIGH, Severity.URGENT

[java.lang.Enum](#) is the implicit supertype of any Enum type

Enums can have some data associated that is given using a private constructor.

Example

```
public enum SmartSeverity {  
    LOW(30), MEDIUM(15), HIGH(7), URGENT(1);  
    private int projectedTurnaroundDays;  
    private SmartSeverity(int projectedTurnaroundDays) {  
        this.projectedTurnaroundDays = projectedTurnaroundDays;  
    }  
  
    public int getProjectedTurnaroundDays() {  
        return projectedTurnaroundDays;  
    }  
}
```

Appendix

Wildcards and Subtyping

Upper bounded wildcard

? extends T

Lower bounded wildcard

? super T

